=== UNIT 3: MODULAR PROGRAMMING: FUNCTIONS, RECURSION & STORAGE CLASSES ===

>> 3.1 Function Architecture, Prototypes and Parameter Passing

Modular programming decomposes complex engineering algorithms into reusable, testable subroutines.
* Anatomy of a Function: Return type, function identifier, formal parameter list, local variable declarations, and function body.
* Function Prototype (Declaration): Informs compiler about function signature (return type, parameter types, count) before actual definition: int calculateArea(int width, int height);. Enables type checking and implicit conversions.
* Call-by-Value vs. Call-by-Reference:
- Call-by-Value: Default in C. Actual arguments are evaluated and values copied into separate formal parameter memory locations. Modifications made inside function do NOT affect caller's actual arguments.
- Call-by-Reference (Simulated via Pointers): Caller passes memory addresses (&var). Formal parameters are declared as pointers (*ptr). Function dereferences pointers to directly mutate caller's original memory.

>> 3.2 Recursion and Call Stack Mechanics

* Recursive Function: A function that calls itself either directly or indirectly. Requires two components: (1) Base Case (terminating condition returning without recursion), (2) Recursive Step (reduces problem instance closer to base case).
* Execution Stack Trace: Each function call allocates an Activation Record (Stack Frame) on call stack containing parameters, local variables, and return address. Once base case is reached, stack frames unwind in Last-In-First-Out (LIFO) order.
* Tail Recursion: Recursive call is tail-recursive if it is the very last operation executed, allowing compiler optimization (Tail Call Optimization TCO) to reuse stack frames and avoid stack overflow.

>> 3.3 Storage Classes and Variable Scope

* auto: Default for local variables declared inside blocks. Stored on Stack. Scope local to block; lifetime ends upon block exit. Default value is garbage.
* register: Requests compiler to store variable in high-speed CPU core register. Cannot use address-of operator (&) on register variables.
* static: Local static variables retain values between successive function calls. Initialized only ONCE at program startup. Stored in Data Segment (BSS if uninitialized). Global static functions/variables have internal linkage (restricted strictly to declaring file).
* extern: Declares a global variable or function defined in another source file.

Possesses external linkage, informing linker to resolve symbol across object files without allocating duplicate memory.

[PROGRAM 1: Call-by-Value vs. Call-by-Reference Demonstration (Swap Function)]

Problem: Write two swap functions in C to rigorously contrast Call-by-Value and Call-by-Reference (via pointers), inspecting memory addresses before, during, and after execution.

Source Code:

```
#include <stdio.h>
void swapByValue(int x, int y) {
int temp = x;
x = y;
y = temp;
printf(" Inside swapByValue: x = %d, y = %d (Formal parameters altered)\n", x, y);
}
void swapByReference(int *x, int *y) {
int temp = *x;
*x = *y;
*y = temp;
printf(" Inside swapByReference: *x = %d, *y = %d (Original memory modified)\n", *x,
*y);
}
int main() {
int a = 10, b = 20;
printf("Initial: a = %d, b = %d\n", a, b);
swapByValue(a, b);
printf("After swapByValue: a = %d, b = %d (UNALTERED)\n", a, b);
swapByReference(&a, &b);
printf("After swapByReference: a = %d, b = %d (SUCCESSFULLY SWAPPED)\n", a, b);
return 0;
}
```

Output:

```
Initial: a = 10, b = 20
Inside swapByValue: x = 20, y = 10 (Formal parameters altered)
After swapByValue: a = 10, b = 20 (UNALTERED)
Inside swapByReference: *x = 20, *y = 10 (Original memory modified)
After swapByReference: a = 20, b = 10 (SUCCESSFULLY SWAPPED)
```

[PROGRAM 2: Recursion vs. Iteration - Factorial & Stack Depth Analysis]

Problem: Implement factorial computation using: (1) Standard recursion, (2)

Tail-recursive optimization, and analyze stack memory overhead.
Source Code:

```
#include <stdio.h>
long long factorialStandard(int n) {
if (n <= 1) return 1;
return n * factorialStandard(n - 1);
}
long long factorialTail(int n, long long accumulator) {
if (n <= 1) return accumulator;
return factorialTail(n - 1, n * accumulator);
}
int main() {
int val = 6;
printf("Standard Factorial(%d): %lld\n", val, factorialStandard(val));
printf("Tail Recursive Factorial(%d): %lld\n", val, factorialTail(val, 1));
return 0;
}
```

Stack Trace for factorialStandard(4):
Call 1: fact(4) -> waits for 4 * fact(3)
Call 2: fact(3) -> waits for 3 * fact(2)
Call 3: fact(2) -> waits for 2 * fact(1)
Call 4: fact(1) -> returns 1 (Base Case reached).
Unwind 3: returns 2 * 1 = 2.
Unwind 2: returns 3 * 2 = 6.
Unwind 1: returns 4 * 6 = 24. Result = 24.

[PROGRAM 3: Tower of Hanoi Recursive Solver]

Problem: Solve the Tower of Hanoi puzzle for N disks moving from source rod A to destination rod C using auxiliary rod B. Calculate total moves: 2^N - 1.
Source Code:

```
#include <stdio.h>
void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod, int *moveCount) {
if (n == 1) {
(*moveCount)++;
printf("Move %2d: Move disk 1 from rod %c to rod %c\n", *moveCount, from_rod, to_rod);
return;
}
towerOfHanoi(n - 1, from_rod, aux_rod, to_rod, moveCount);
(*moveCount)++;
printf("Move %2d: Move disk %d from rod %c to rod %c\n", *moveCount, n, from_rod,
to_rod);
towerOfHanoi(n - 1, aux_rod, to_rod, from_rod, moveCount);
}
```

```
int main() {
int n = 3, count = 0;
printf("Tower of Hanoi Solution for %d disks (Total moves = %d):\n", n, (1 << n) - 1);
towerOfHanoi(n, ’A’, ’C’, ’B’, &count);
return 0;
}
```

Output Sequence (7 moves):
Move 1: Move disk 1 from rod A to rod C
Move 2: Move disk 2 from rod A to rod B
Move 3: Move disk 1 from rod C to rod B
Move 4: Move disk 3 from rod A to rod C
Move 5: Move disk 1 from rod B to rod A
Move 6: Move disk 2 from rod B to rod C
Move 7: Move disk 1 from rod A to rod C

[PROGRAM 4: Static Local Variable Behavior Across Repeated Invocations]

Problem: Write a program demonstrating how static local variables maintain state across multiple independent function calls, contrasting with standard automatic variables.
Source Code:

```
#include <stdio.h>
void transactionLogger() {
auto int autoCounter = 0;
static int staticCounter = 0;
autoCounter++;
staticCounter++;
printf("Call Output: autoCounter = %d | staticCounter = %d\n", autoCounter,
staticCounter);
}
int main() {
printf("Invoking transactionLogger() 4 times:\n");
for (int i = 0; i < 4; i++) {
transactionLogger();
}
return 0;
}
```

Output:
Call 1: autoCounter = 1 | staticCounter = 1
Call 2: autoCounter = 1 | staticCounter = 2
Call 3: autoCounter = 1 | staticCounter = 3
Call 4: autoCounter = 1 | staticCounter = 4
Conclusion: Static variables persist in Data Segment across entire program lifetime while retaining local block scope.

>> 3.4 University Review & Examination Questions

* Q1: Contrast all four C storage classes (auto, register, static, extern) across storage location, default initial value, scope, and lifetime.
Answer: (1) auto: Storage: RAM (Stack); Default Value: Garbage; Scope: Local to declaring block; Lifetime: Duration of block execution. (2) register: Storage: CPU core register; Default Value: Garbage; Scope: Local to block; Lifetime: Duration of block execution; Cannot take address (&). (3) static: Storage: RAM (Data/BSS segment); Default Value: Zero (0); Scope: Local to block (or file scope if global); Lifetime: Entire program execution lifetime; retains values between function invocations. (4) extern: Storage: RAM (Data/BSS segment); Default Value: Zero (0); Scope: Global (accessible across multiple project files); Lifetime: Entire program execution lifetime.
* Q2: Explain the mechanism of recursive function execution using the activation record and call stack concept.
Answer: When a function is called, the runtime environment allocates an Activation Record (Stack Frame) on top of the program's call stack. The frame contains: (1) Return address, (2) Storage for actual arguments, (3) Storage for local variables, (4) Saved machine registers. In recursion, each recursive invocation pushes a NEW, independent stack frame onto the call stack. These frames accumulate during the winding phase. If base case is absent or incorrect, stack memory exhausts, causing fatal Stack Overflow. When base case is reached, function completes and returns value; its stack frame is popped off, and control transfers back to caller frame. Unwinding phase continues until initial caller receives final accumulated result.

>> 3.5 Key Takeaways and Revision Points

* C uses call-by-value by default; call-by-reference is simulated by passing pointer addresses.
* Recursion requires a strictly reachable base case to prevent stack overflow.
* static variables retain their values across function calls; stored in data segment.
* extern enables sharing global variables across multiple compilation units.